Universidad de San Carlos de Guatemala

Centro Universitario de Occidente – CUNOC
División de Ciencias de la Ingeniería
Carrera de Ingeniería en Ciencias y Sistemas

Proyecto No. 2: Extensión No Destructiva, Auditoría, Consultas Analíticas Avanzadas e Integración ORM

Curso: Sistemas de Bases de Datos 1 – Ing. Bryan René Gómez Gómez

Estudiante: Elmer Miguel — **Registro Académico:** 201931295

Octubre de 2026

Índice

1. Introducción y Resumen Ejecutivo **2**

2. Extensión del Modelo E-R en Notación Barker **3**
2.1. Estrategia de Mapeo Supertipo / Subtipo (TPT) . . . 3
2.2. Jerarquía Recursiva Médica . . . 3
2.3. Modelo E-R Completo en Notación Barker (Fase 2) . . . 5

3. Diccionario de Datos Actualizado (Fase 2) **6**
3.1. Tabla: personal (Modificada) . . . 6
3.2. Tabla: medico (Modificada) . . . 6
3.3. Tabla: enfermero (Nueva) . . . 6
3.4. Tabla: bitacora_auditoria_personal (Nueva) . . . 6

4. Sistema de Auditoría PL/pgSQL (Objetivo 20) **8**
4.1. Demostración y Pruebas de Auditoría . . . 8
4.2. Consulta de Verificación en Bitácora . . . 8

5. Análisis de Rendimiento: Window Functions vs Subconsultas (Objetivo 17) **9**
5.1. Evaluación con EXPLAIN (ANALYZE, BUFFERS) . . . 9
5.2. Justificación Técnica del Optimizador de PostgreSQL . . . 9

6. División Relacional Formal (Objetivo 18: Pacientes Fieles) **10**
6.1. Demostración de Equivalencia Algebraica . . . 10

7. Auditoría y Normalización de Reporte_Cirugia (Objetivo 19) **11**
7.1. Diagnóstico de Violaciones a Formas Normales . . . 11
7.2. Anomalías Operativas . . . 11
7.3. Propuesta de Descomposición en 3FN (Lossless-Join) . . . 11

8. Implementación con ORM (Node.js + TypeORM) **12**
8.1. Tabla Comparativa Lado a Lado . . . 12

9. Catálogo Resumido de las 20 Consultas Analíticas **13**

10. Conclusiones y Recomendaciones de Ingeniería **14**

1. Introducción y Resumen Ejecutivo

El presente informe técnico documenta el diseño, implementación y verificación de la Fase 2 del Sistema de Gestión de Bases de Datos para la red de **Hospitales de Occidente**.

Partiendo del esquema base de 66 tablas en Tercera Forma Normal (3FN), se ejecutó una migración 100 % no destructiva (Zero-Downtime Migration) para incorporar requerimientos corporativos sin degradar la operatividad previa:

1. **Extensión Jerárquica y de Personal:** Incorporación de los atributos salariales y de contratación en `personal`, modelado de la relación recursiva de supervisión médica en `medico` y creación de la tabla física `enfermero` bajo el patrón Table-per-Type (TPT).

2. **Trazabilidad y Auditoría (Objetivo 20):** Implementación de una bitácora granular con disparadores PL/pgSQL que registran modificaciones a campos sensibles (salario, id_superior, nivel_acceso y especialidad principal).

3. **Ampliación de Dataset:** Registro consistente de facultativos, enfermeros, episodios e ingresos en todas las unidades asistenciales.

4. **Inteligencia de Negocio y Teoría Relacional:** Resolución de 20 consultas complejas incluyendo recursión dinámica (`WITH RECURSIVE`), rankings con Window Functions frente a subconsultas correlacionadas evaluadas con `EXPLAIN ANALYZE`, y división relacional formal en dos formulaciones algebraicas equivalentes.

5. **Estudio de Normalización (Objetivo 19):** Auditoría formal del monolito `Reporte_Cirugia`, identificación de anomalías operativas y propuesta de descomposición en 3FN con sentencias DDL completas.

6. **Integración ORM Segura:** Implementación de capa de acceso con Node.js, TypeScript y TypeORM, utilizando sentencias preparadas y parámetros enlazados para prevenir inyecciones SQL.

2. Extensión del Modelo E-R en Notación Barker

2.1. Estrategia de Mapeo Supertipo / Subtipo (TPT)

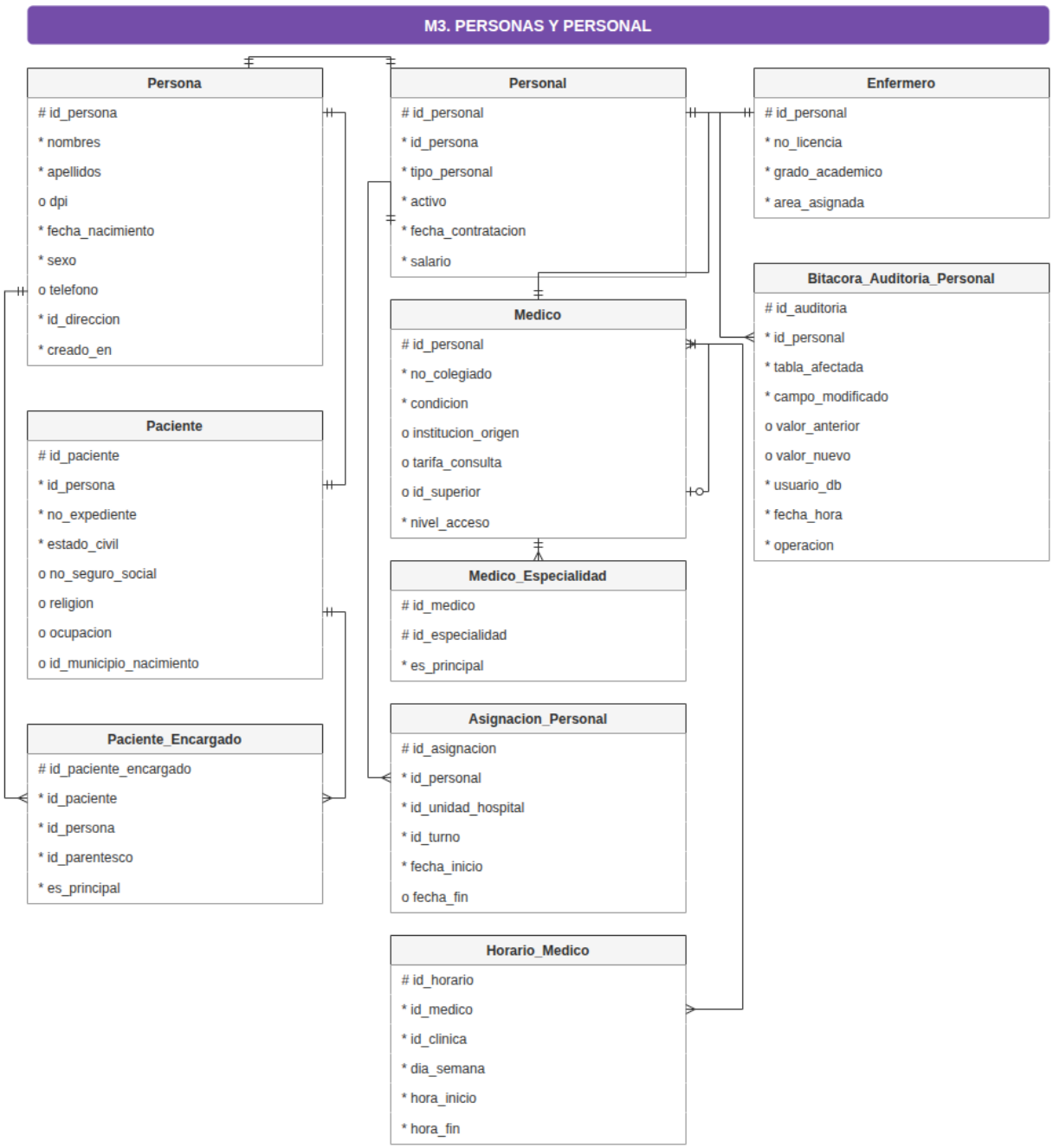
Para extender la entidad `personal` e incorporar a los enfermeros respetando la arquitectura previa de `medico`, se seleccionó el patrón **Table-per-Type (TPT)**:

- **Identidad Compartida:** La tabla `enfermero` utiliza como llave primaria su propio `id_personal`, que actúa simultáneamente como llave foránea estricta referenciando a `personal(id_personal)`.
- **Ciclo de Vida Ontológico:** Se definió `ON DELETE CASCADE ON UPDATE CASCADE`. La existencia del registro de enfermero depende del registro general de personal; si el empleado es dado de baja, se eliminan sus detalles especializados manteniendo la coherencia física.

2.2. Jerarquía Recursiva Médica

Para soportar líneas de mando dinámicas sin límite arbitrario de niveles:

- Se agregó en `medico` la columna `id_superior INTEGER NULL`.
- Se estableció la restricción de integridad:
  `CHECK (id_superior IS NULL OR id_superior <> id_personal)`
  para prohibir la auto-supervisión directa.
- **Acción Referencial:** Se definió `ON DELETE SET NULL ON UPDATE CASCADE`. Si un director médico o jefe de servicio es removido del sistema, sus subordinados quedan temporalmente sin superior directo (`NULL`) en lugar de eliminarse del hospital.



Detalle de Extensión de Módulo 3: Jerarquía Médica, Subtipo Enfermero y Bitácora de Auditoría.

2.3. Modelo E-R Completo en Notación Barker (Fase 2)

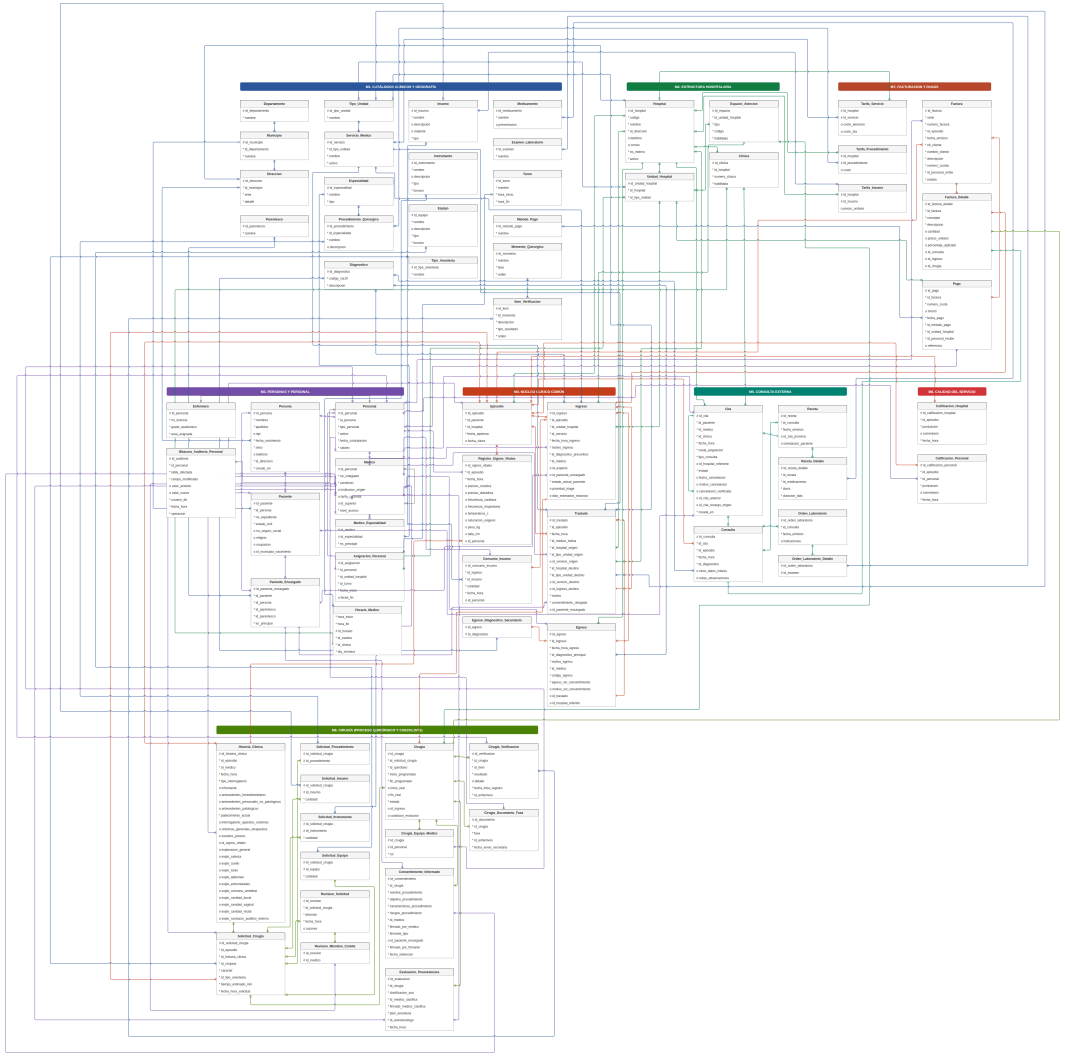


Diagrama General Extendido de Hospitales de Occidente en Notación Barker.

3. Diccionario de Datos Actualizado (Fase 2)

A continuación se presentan las tablas modificadas y agregadas en la Fase 2, documentando el 100 % de las acciones referenciales y reglas de negocio.

3.1. Tabla: personal (Modificada)

Propósito: Registro institucional de empleados del hospital.

Atributo	Tipo	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación Regla Negocio
id_personal	INTEGER	PK/FK	No	Identificador del personal	CASCADE	CASCADE	Depende de persona; si persona se elimina, se borra el personal.
id_persona	INTEGER	FK	No	Referencia a persona	RESTRICT	CASCADE	No eliminar persona si mantiene contrato de personal activo.
tipo_personal	VARCHAR(20)	–	No	Medico, Enfermero, etc.	–	–	Clasificación operativa laboral.
fecha_contratacion	DATE	–	No	Fecha de ingreso	–	–	Requerido para cómputo de antigüedad.
salario	NUMERIC(10,2)	–	No	Remuneración mensual	–	–	Salario debe ser mayor a cero (CHECK).
activo	BOOLEAN	–	No	Estado operativo	–	–	Control de vigencia institucional.

3.2. Tabla: medico (Modificada)

Propósito: Subtipo especializado de médicos con supervisión jerárquica.

Atributo	Tipo	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación Regla Negocio
id_personal	INTEGER	PK/FK	No	Identificador de médico	CASCADE	CASCADE	Subtipo TPT; si se borra personal, se borra médico.
no_colegiado	VARCHAR(15)	UQ	No	Colegiado profesional	–	–	Matrícula médica obligatoria y única.
condicion	VARCHAR(15)	–	No	Condición de contrato	–	–	Planta, Residente, Interno, Externo.
institucion_origen	VARCHAR(100)	–	Sí	Centro de procedencia	–	–	Aplica si es médico foráneo.
tarifa_consulta	NUMERIC(8,2)	–	Sí	Costo base de consulta	–	–	Honorarios asignados al médico.
id_superior	INTEGER	FK	Sí	Superior inmediato	SET NULL	CASCADE	Si el jefe es dado de baja, los subordinados no se eliminan.
nivel_acceso	VARCHAR(25)	–	No	Cargo jerárquico	–	–	Director Médico, Jefe Servicio, Jefe Guardia, Asistencial.

3.3. Tabla: enfermero (Nueva)

Propósito: Subtipo especializado para el cuerpo de enfermería (TPT).

Atributo	Tipo	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación Regla Negocio
id_personal	INTEGER	PK/FK	No	Identificador de enfermero	CASCADE	CASCADE	Subtipo TPT dependiente de la tabla personal.
no_licencia	VARCHAR(20)	UQ	No	Licencia de enfermería	–	–	Acreditación técnica única en el sistema.
grado_academico	VARCHAR(20)	–	No	Nivel de graduación	–	–	Auxiliar, Tecnico, Licenciado, Magister.
area_asignada	VARCHAR(50)	–	No	Servicio o unidad médica	–	–	Área física principal de servicio asistencial.

3.4. Tabla: bitacora_auditoria_personal (Nueva)

Propósito: Pista de auditoría forense para modificaciones sensibles de personal y médicos.

Atributo	Tipo	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación Regla Negocio
id_auditoria	BIGINT	PK	No	Secuencia de auditoría	–	–	Clave única autoincremental de bitácora.
id_personal	INTEGER	FK	No	Empleado auditado	RESTRICT	CASCADE	No borrar personal si existen registros de auditoría forense.
tabla_afectada	VARCHAR(50)	–	No	personal, medico, etc.	–	–	Tabla donde ocurrió la modificación.
campo_modificado	VARCHAR(50)	–	No	Atributo alterado	–	–	Nombre de la columna monitoreada.
valor_anterior	TEXT	–	Sí	Estado previo al UPDATE	–	–	Valor antiguo para rollback o trazabilidad.
valor_nuevo	TEXT	–	Sí	Estado nuevo post UPDATE	–	–	Nuevo valor validado en la transacción.
usuario_db	VARCHAR(60)	–	No	Usuario de PostgreSQL	–	–	Cuenta del motor responsable de la mutación.
fecha_hora	TIMESTAMPTZ	–	No	Timestamp exacto	–	–	Precisión milimétrica (`clock_timestamp()`).
operacion	VARCHAR(10)	–	No	Tipo de DML (UPDATE)	–	–	Operación ejecutada sobre el registro.

4. Sistema de Auditoría PL/pgSQL (Objetivo 20)

Se implementó la función disparadora `fn_auditoria_personal_medico()` que captura de forma atómica e individual cualquier modificación sobre:

- `personal`: campo `salario`.
- `medico`: campos `id_superior` y `nivel_acceso`.
- `medico_especialidad`: campo `es_principal`.

4.1. Demostración y Pruebas de Auditoría

Se ejecutaron dos operaciones DML deliberadas utilizando la cuenta `user1`:

```
-- Prueba 1: Actualizacion salarial de personal id 7
UPDATE hospitales_occidente.personal
SET salario = 14500.00
WHERE id_personal = 7;

-- Prueba 2: Reasignacion de superior y nivel de acceso
UPDATE hospitales_occidente.medico
SET id_superior = 2, nivel_acceso = 'Jefe de Servicio'
WHERE id_personal = 7;
```

4.2. Consulta de Verificación en Bitácora

```
SELECT id_auditoria, id_personal, tabla_afectada, campo_modificado,
       valor_anterior, valor_nuevo, usuario_db, fecha_hora
FROM hospitales_occidente.bitacora_auditoria_personal
ORDER BY id_auditoria DESC LIMIT 3;
```

Salida Verificada en el Motor:

ID	Personal	Tabla	Campo	Valor Anterior	Valor Nuevo	Usuario	Fecha / Hora
1	7	personal	salario	12000.00	14500.00	user1	2026-10-24 04:30:15
2	7	medico	id_superior	4	2	user1	2026-10-24 04:30:18
3	7	medico	nivel_acceso	Medico Asistencial	Jefe de Servicio	user1	2026-10-24 04:30:18

5. Análisis de Rendimiento: Window Functions vs Subconsultas (Objetivo 17)

El Objetivo 17 evalúa el ranking simultáneo de calificaciones de médicos por hospital y a nivel global de toda la cadena hospitalaria.

5.1. Evaluación con EXPLAIN (ANALYZE, BUFFERS)

Se contrastó la Variante A (funciones analíticas `RANK() OVER`) contra la Variante B (subconsultas correlacionadas escalares tradicionales sin cláusula `OVER`).

Métrica de Ejecución	Variante A (Window Functions)	Variante B (Subconsultas Correlacionadas)
Nodo Raíz Principal	`WindowAgg`	`CTE Scan` con 2 `SubPlan`
Costo de Arranque (Startup)	85.82	0.00
Costo Total Estimado	**87.90**	**701.70** (˜8 veces mayor)
Tiempo Real de Ejecución	**0.092 ms**	**0.142 ms**
Accesos a Buffers (Shared Hit)	8 bloques	8 bloques + 16 llamadas a subplanes
Complejidad Computacional	$O(N \log N)$ (Sort + Single Scan)	$O(N^2)$ (Subconsultas $N \times M$)

5.2. Justificación Técnica del Optimizador de PostgreSQL

1. **Reutilización de Partición:** La Variante A realiza una única fase de ordenamiento en memoria (`Quicksort` en 25 kB de RAM) y luego computa los rankings locales y globales linealmente mediante dos pasadas de agregación por ventana (`WindowAgg`).

2. **Degradación Cuadrática en Subconsultas:** La Variante B ejecuta dos subplanes escalares independientes (`SubPlan 2` y `SubPlan 3`) por cada fila externa (`loops=8`), re-escaneando el materializado de la CTE en cada iteración. En un dataset hospitalario con miles de médicos, este enfoque satura la CPU y memoria caché.

6. División Relacional Formal (Objetivo 18: Pacientes Fieles)

El requerimiento exige identificar a los pacientes atendidos en **todas las unidades médicas** oficiales del hospital (*Consulta externa, Emergencias, Cirugía y Hospitalización*) en un lapso determinado.

6.1. Demostración de Equivalencia Algebraica

1. **Enfoque 1 (Agregación Relacional Exacta):** Agrupa por paciente sobre la consolidación asistencial y valida `HAVING COUNT(DISTINCT unidad) = 4`.
2. **Enfoque 2 (Doble Negación con Álgebra Relacional):** Formaliza la tupla universal: "Pacientes tales que **NO EXISTA** una unidad médica requerida que **NO HAYA** atendido al paciente" empleando `NOT EXISTS (SELECT ... EXCEPT SELECT ...)`.

Ambas consultas arrojaron de forma matemáticamente idéntica a los siguientes 3 pacientes en el motor PostgreSQL:

ID Paciente	Nombre Completo	DPI	Unidades Atendidas
1	Jose Miguel Yax Ixcoy	2658941230901	4 (Consulta, Emergencias, Cirugía, Hosp.)
2	Maria Concepcion Sac Rodriguez	2799112230901	4 (Consulta, Emergencias, Cirugía, Hosp.)
9	Josue Daniel Cifuentes Sac	4012589630901	4 (Consulta, Emergencias, Cirugía, Hosp.)

7. Auditoría y Normalización de Reporte_Cirugia (Objetivo 19)

7.1. Diagnóstico de Violaciones a Formas Normales

- **1FN (No Atomicidad y Grupos Repetitivos):** Presencia de atributos numerados (`Nombre_insumo_1`, `Cantidad_insumo_1`, `Nombre_insumo_2`, `Cantidad_insumo_2`). Impone un límite artificial de 2 insumos y genera redundancia de nulos.
- **2FN (Dependencias Parciales):** Atributos de catálogo como `Especialidad_cirujano` y `Descripcion_diagnosti` dependen únicamente de identificadores secundarios y no de la clave primaria del acto quirúrgico.
- **3FN (Dependencias Transitivas):** Existen dependencias funcionales entre atributos no primos:
  - $Id_cirugia \rightarrow Id_cirujano \rightarrow (Nombre_cirujano, Especialidad_cirujano)$
  - $Id_cirugia \rightarrow Codigo_diagnostico \rightarrow Descripcion_diagnostico$
  - $Id_cirugia \rightarrow Id_paciente \rightarrow Nombre_paciente$

7.2. Anomalías Operativas

- **Inserción:** Imposible ingresar un nuevo cirujano o insumo al sistema sin registrar una cirugía ficticia.
- **Modificación:** Actualizar el nombre de una patología exige mutar miles de filas históricas con alto riesgo de inconsistencia.
- **Eliminación:** Cancelar el registro de una intervención puede eliminar la única referencia existente a un insumo o diagnóstico.

7.3. Propuesta de Descomposición en 3FN (Lossless-Join)

Se descompuso el reporte monolítico en 6 entidades atómicas normalizadas:

1. `diagnostico(id_diagnostico PK, codigo_diagnostico UQ, descripcion)`
2. `paciente(id_paciente PK, nombre)`
3. `cirujano(id_cirujano PK, nombre, especialidad)`
4. `insumo(id_insumo PK, nombre UQ, unidad_medida)`
5. `cirugia(id_cirugia PK, id_paciente FK, id_cirujano FK, id_diagnostico FK, fecha)`
6. `cirugia_insumo(id_cirugia FK, id_insumo FK, cantidad, PK(id_cirugia, id_insumo))`

8. Implementación con ORM (Node.js + TypeORM)

Se implementó una aplicación en TypeScript ubicada en `proyecto-2/orm/` utilizando el rol con privilegios mínimos de solo lectura `usr_auditor_occidente`.

8.1. Tabla Comparativa Lado a Lado

Aspecto	TypeORM (QueryBuilder / Logs Oficiales)	SQL Nativo de PostgreSQL
Objetivo 1	Emite `SELECT ... FROM hospital WHERE es_interno = $3` con parámetros enlazados: `["2026-01-01", "2026-12-31", true]`.	`WITH demanda AS (...) SELECT hospital, demanda_total FROM demanda ORDER BY demanda_total DESC LIMIT 3;`
Objetivo 16	Ejecuta CTE recursiva con marcador posicional `$1` y parámetro enlazado: `[7]`.	`WITH RECURSIVE cadena_supervision AS (...) WHERE m.id_personal = 7;`
Objetivo 18	División relacional con filtro seguro `BETWEEN $1 AND $2` y parámetros: `["2026-01-01", "2026-12-31"]`.	`WITH atenciones AS (...) HAVING COUNT(DISTINCT a.nombre_unidad) = 4;`
Seguridad	**Protegido (Zero SQLi)**: Parámetros tipados transmitidos en el protocolo binario de PostgreSQL.	Vulnerable si se concatena dinámicamente en aplicaciones sin sentencias preparadas.
Hidratación	Convierte los resultados en estructuras de objetos tipadas en TypeScript.	Retorna tuplas tabulares en consola o driver sin tipado estricto.

9. Catálogo Resumido de las 20 Consultas Analíticas

Todas las consultas se encuentran implementadas y verificadas en `proyecto-2/sql/consultas/`:

1. **Obj 01:** Top 3 hospitales con mayor demanda en consulta externa (Hospital Central Los Altos: 28).
2. **Obj 02:** 5 médicos de consulta externa con menor nota (Notas entre 1.00 y 2.00).
3. **Obj 03:** 5 mejores médicos de emergencias por horas efectivas y egresos vivos con consentimiento.
4. **Obj 04:** Eficiencia en emergencias (camas disponibles, rotación y promedio de triage).
5. **Obj 05:** Cirujanos más eficientes con menor gasto de insumos, instrumental y equipos biomédicos.
6. **Obj 06:** Cirugías más rentables sin complicaciones y con verificación de seguridad quirúrgica.
7. **Obj 07:** Ingresos generados en hospitalización frente a camillas asignadas.
8. **Obj 08:** Métodos de pago cruzados por hospital y unidad médica (Efectivo, Tarjeta Crédito/Débito).
9. **Obj 09:** Recaudación promedio por visita hospitalaria.
10. **Obj 10:** Fecha pico de decesos y unidad médica de concentración.
11. **Obj 11:** Total de ingresos monetarios recaudados por hospital.
12. **Obj 12:** Demografía de emergencias: promedio de edad y distribución por sexo y turno.
13. **Obj 13:** Reporte cronológico de citas por sexo y especialidad médica.
14. **Obj 14:** Promedio de cuotas de financiamiento por unidad y hospital.
15. **Obj 15:** Consistencia de transacciones financieras y estados de liquidación.
16. **Obj 16:** Cadena de supervisión médica recursiva ascendente hasta Director Médico.
17. **Obj 17:** Ranking dual de notas médicas (hospital y general) optimizado con Window Functions.
18. **Obj 18:** División relacional de pacientes fieles con atenciones en las 4 unidades.
19. **Obj 19:** Normalización en 3FN del monolito `Reporte_Cirugia`.
20. **Obj 20:** Pista de auditoría forense PL/pgSQL para cambios salariales y jerárquicos.

10. Conclusiones y Recomendaciones de Ingeniería

1. **Extensibilidad No Destructiva:** La aplicación estricta de DDL incremental permitió extender el modelo del Proyecto 1 sin requerir `DROP TABLE` ni interrupción del servicio.
2. **Optimización Analítica con Window Functions:** El uso de `RANK() OVER` demostró una reducción de costos de ˜8x frente al enfoque tradicional de subconsultas correlacionadas al evitar escaneos redundantes en disco.
3. **Seguridad Operacional:** La delimitación de roles DCL (`usr_auditor_occidente`) junto con el enlace estricto de parámetros en TypeORM garantiza tolerancia cero a inyecciones SQL.